

AWS Image Captioning Web Application

Portfolio Case Study

Albert Huang

Deployment status: This project was deployed in an AWS Academy sandbox environment. The live AWS resources are no longer available because the sandbox has expired. This document is a portfolio case study containing architecture diagrams, implementation notes, and project evidence. Credentials, personal identifiers, account-specific endpoints, and sensitive configuration details have been removed or generalised.

1. Project Overview

This project demonstrates the deployment of a cloud-based image annotation web application on AWS. The system combines a Flask web application, scalable EC2 infrastructure, S3 object storage, RDS MySQL metadata storage, and event-driven serverless processing with AWS Lambda and Amazon EventBridge.

Users upload images through the Flask application. Uploaded images are stored in Amazon S3, then S3 object-created events are routed through EventBridge to trigger two Lambda functions. One function generates image thumbnails and stores them back in S3; the other calls the Gemini REST API to generate captions and inserts the caption metadata into Amazon RDS MySQL.

Technology Stack

Area	Technologies / Services
Backend application	Python, Flask, HTML templates
Cloud infrastructure	Amazon EC2, Application Load Balancer, Auto Scaling Group, VPC, public/private subnets, NAT Gateway, Internet Gateway
Storage and database	Amazon S3, Amazon RDS MySQL
Serverless processing	AWS Lambda, Amazon EventBridge, boto3, PIL/Pillow
AI integration	Gemini REST API
Security and administration	IAM role permissions, Security Groups, bastion host, environment variables

2. Architecture Overview

The project uses two connected architectures: a scalable web application architecture and an event-driven serverless architecture. The web layer provides user access and retrieves stored results, while the serverless layer performs asynchronous processing after new S3 objects are created.

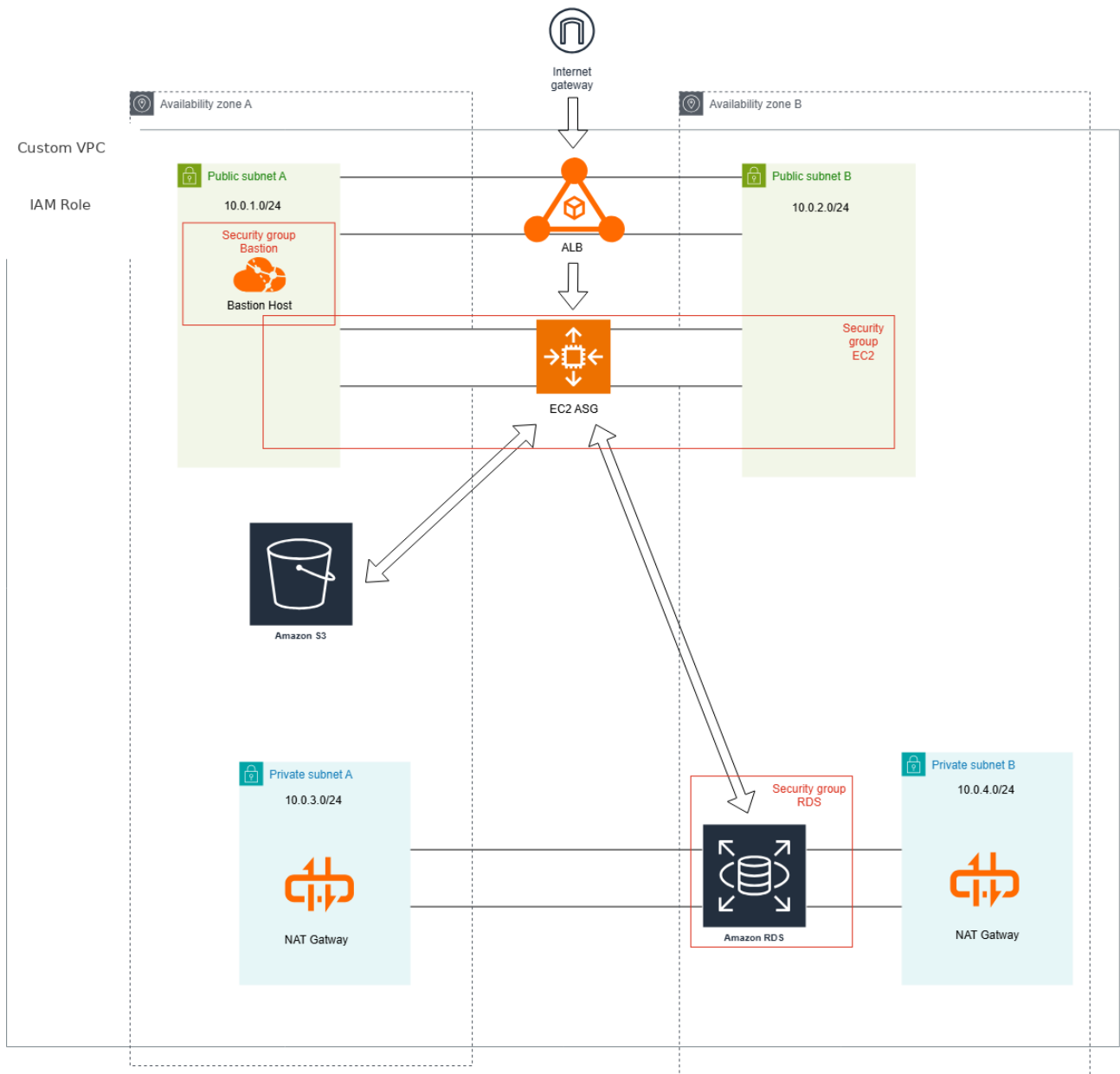


Figure 1. Web application architecture with ALB, EC2 Auto Scaling, S3, RDS, public/private subnets, NAT Gateway, and bastion access. Resource names are illustrative and account-specific details have been omitted.

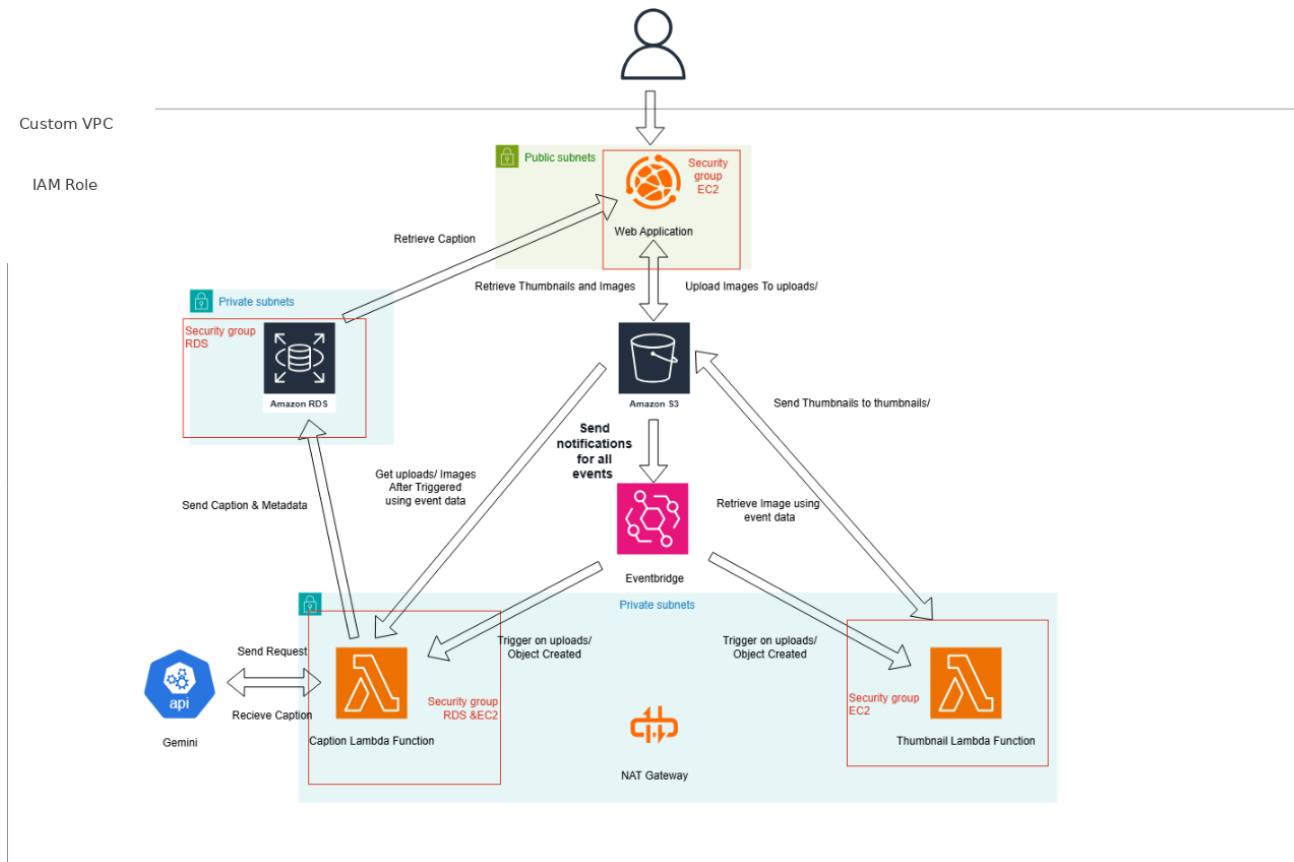


Figure 2. Serverless workflow: S3 object-created events are routed through EventBridge to the caption and thumbnail Lambda functions.

Component Interaction

- Users access the Flask web application through an Application Load Balancer. EC2 instances are managed by an Auto Scaling Group to provide horizontal scalability and improved availability.
- The application stores original uploaded images under the S3 uploads/ prefix and retrieves generated thumbnails from the thumbnails/ prefix.
- S3 events are sent to EventBridge. An EventBridge rule filters for ObjectCreated events under uploads/ and triggers both Lambda functions.
- The caption Lambda function retrieves the uploaded image, calls the Gemini REST API, and writes the image key and generated caption to RDS MySQL.
- The thumbnail Lambda function retrieves the uploaded image, creates a 128 x 128 thumbnail using PIL/Pillow, and stores the thumbnail in S3.

3. Web Application Deployment

Compute Environment

The Flask application was hosted on EC2 instances launched from a common launch template. The template used Amazon Linux 2023 and a small instance type suitable for the AWS Academy sandbox. A user data script installed operating system packages and Python dependencies, downloaded the application deployment package from S3, and started the Flask application.

An Application Load Balancer listened on HTTP port 80 and forwarded traffic to a target group containing EC2 instances managed by an Auto Scaling Group. Health checks were configured against the application root path so unhealthy instances could be removed from rotation and replaced automatically.

Network and Administration

- A custom VPC was created with public and private subnets across two Availability Zones.
- Public subnets hosted internet-facing components such as the ALB and bastion host.
- Private subnets hosted application/database-related resources that should not be directly exposed to the public internet.

- A NAT Gateway allowed private subnet resources, including Lambda functions running in the VPC, to reach external services such as the Gemini API.
- Administrative SSH access was performed through a bastion host, reducing direct exposure of internal resources.

4. Storage and Database Design

Amazon S3 Storage

A general purpose S3 bucket was used for three categories of project files: application deployment assets, original uploaded images, and generated thumbnails. Uploaded images were stored under uploads/, while thumbnails were stored under thumbnails/. Separating these prefixes was important because it allowed EventBridge filtering to trigger Lambda only for original uploads and avoid recursive triggering when thumbnails were written back to S3.

Amazon RDS MySQL

Amazon RDS MySQL stored image metadata and captions in a database table. The Flask application retrieved image keys and captions for display, while the caption Lambda function inserted new caption records after calling the Gemini API.

Representative schema: captions(id, image_key, caption, uploaded_at)

5. Event-Driven Serverless Workflow

The project used EventBridge because one uploaded image needed to trigger two independent processing tasks. This design separates image upload handling from asynchronous post-processing and keeps the web application responsive.

- S3 EventBridge notifications were enabled for the bucket.
- An EventBridge rule filtered for ObjectCreated events with the uploads/ prefix.
- The rule targeted both the caption Lambda function and the thumbnail Lambda function.
- Both Lambda functions performed additional runtime checks to ignore non-image files and skip objects under thumbnails/.

Caption Lambda Function

- Parsed the EventBridge event envelope to extract the bucket name and object key.
- Downloaded the uploaded image from S3 and base64-encoded the image payload.
- Called the Gemini REST API to generate a natural-language caption.
- Parsed the API response and inserted the image key and caption into the RDS MySQL captions table.
- Used environment variables for external API and database configuration. Sensitive values are excluded from this public case study.

Thumbnail Lambda Function

- Parsed the EventBridge event envelope to identify the uploaded object.
- Downloaded the image from S3 using boto3.
- Generated a 128 x 128 thumbnail using PIL/Pillow.
- Uploaded the generated thumbnail to the thumbnails/ prefix in S3.
- Used prefix checks to avoid recursive processing of generated thumbnails.

6. Security Design

The project followed a basic least-exposure security model appropriate for a sandbox cloud deployment. Public access was limited to the ALB and controlled administrative entry points, while database access was restricted by network placement and security group rules.

- RDS was placed in private subnets rather than exposed publicly.
- The RDS security group allowed MySQL traffic on port 3306 only from the application EC2 security group and the bastion security group.
- Administrative SSH access was restricted through a bastion host rather than opened broadly to application instances.
- Lambda functions used IAM role permissions and environment variables for service access and configuration.
- S3 prefixes separated original uploads from generated thumbnails to prevent recursive event processing.

7. Testing and Observations

The deployment was tested by manually validating the Flask application, checking S3 upload behavior, confirming Lambda triggers, and verifying that generated captions and thumbnails were available to the web application. Troubleshooting was

performed by SSH access through the bastion host, inspecting EC2 files and dependencies, manually running the Flask application, and reviewing Lambda logs/configuration.

Auto Scaling behavior was explored using ApacheBench to generate repeated HTTP GET requests against the ALB endpoint. This test focused on web-layer scaling and did not trigger the image-processing Lambda functions because it did not upload new images.

8. Limitations and Future Improvements

Current Limitations

- The live AWS Academy sandbox environment has expired, so the original deployment is no longer accessible.
- Credentials, API keys, exact endpoints, account-specific resource identifiers, and personal identifiers have been removed.
- The infrastructure was configured primarily through the AWS Console and documented manually, rather than fully automated as infrastructure as code.
- The repository version is intended as a portfolio case study and may include representative snippets rather than a complete redeployable production package.

Future Improvements

- Rebuild the infrastructure using Terraform or CloudFormation for reproducible deployment.
- Add CI/CD automation for application and Lambda updates.
- Add authentication, user-level image ownership, and stronger file validation.
- Add CloudWatch dashboards, structured logging, retry handling, and alerting.
- Move secret management fully into AWS Secrets Manager or Parameter Store.

9. Lessons Learned

This project strengthened practical understanding of AWS web application deployment, event-driven serverless design, VPC networking, security group configuration, database integration, and cloud debugging. Key lessons included the importance of matching Lambda runtime versions with packaged dependencies, increasing Lambda timeout values for API/database operations, using prefix filtering to avoid recursive event triggers, and validating EC2 startup scripts directly when deployment automation does not behave as expected.